

Module 3

Interaction Design Requirements: Extracting Interaction Design Requirements, Constructing Design - Informing Models: second span of the bridge, Some general “how to” suggestions, A New example domain: slideshow presentations, User models, Usage models, Work environment models, Barrier summaries, Model Consolidation, Protecting your sources, Abridged methods for design-informing models extraction, Roots of essential use cases in software use cases.

Interaction Design Requirements

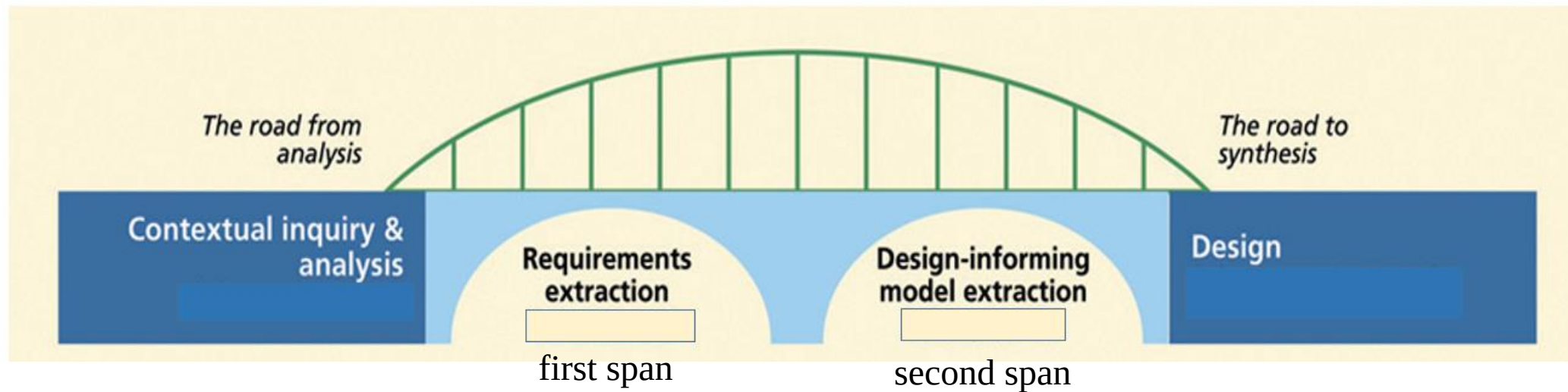
- **Interaction Design Requirements** are a crucial step in the design process.
- They define **what a system or product needs to accomplish to meet the needs of its users and to function effectively in its intended environment.**
- This process focuses on:
 - **identifying user needs,**
 - **understanding the task context, and**
 - **determining the functional and non-functional criteria for the design.**
- This involves carefully examining the Work Activity Affinity Diagram (WAAD) and any preliminary design-informing models, such as the flow model.
- Through deductive reasoning, user needs and requirements can be extracted to construct the first span of the bridge between analysis and design.

Extracting Interaction Design Requirements

Bridging the gap between Analysis and Design:

- Information coming from contextual studies describes the work domain but does not directly meet the information needs in design.
- The output of contextual inquiry and analysis does not speak directly to what is needed as inputs to design. There is a gap.
 - Information coming from contextual studies describes the work domain but does not directly meet the information needs in design.
 - There is a cognitive shift between analysis-oriented thinking on one side of the gap and design-oriented thinking on the other.
 - The gap is the demarcation between the old and the new—between studying existing work practice and existing systems and envisioning a new work space and new system design space.

Overview of the bridge to design



The image uses a **bridge analogy** to represent the transition from understanding a problem to designing a solution.

1. The Road from Analysis:

- This is the starting point where the process focuses on **understanding the context and analyzing the problem**.
- Observing and interacting with users or stakeholders to gather insights about their behaviors, needs, and pain points.
- Analyzing this data to create a thorough understanding of the current situation.

2. The Bridge:

- It is divided into two key components, which are **essential for translating insights into actionable outputs**:
 - **Requirements Extraction:** Identifies and organizes the **functional and non-functional requirements** that need to be addressed.
 - **Design-Informing Model Extraction:** Creating **abstract models or frameworks** based on the analysis to inform the design process.

3. The Road to Synthesis:

- This phase represents **design and creation** based on the insights and models developed in the previous steps.
- The focus here is on **solution-building**, and it is labeled as "Design."
- Activities in this phase include:
 - Concept generation.
 - Prototyping and iteration.
 - Testing solutions to refine and validate them against user needs.
- These two components (requirements and models) form the **pillars of the bridge**, enabling a smooth transition from understanding to ideating solutions.

Requirements Extraction- first span of bridge

- The term refers to **what is needed to design a system that will fulfill user and customer goals.**
- In the UX domain, **interaction design requirements** describe what is required to support user or customer work activity needs.
- We are also concerned with **functional requirements** to ensure the usefulness component of the user experience.

Formal Requirements Extraction

- This process of extracting needs and requirements involves a group sitting down together and going over a large amount of data, including the WAAD and evolving design-informing models.

- Here we perform the following:

1. Walking the WAAD for Needs and Requirements:

- Traverse the hierarchical WAAD structure and focus on extracting requirement statements from work activity notes.

2. Switching from Inductive to Deductive Reasoning:

- Extracting requirements from the WAAD calls for a deductive thinking process.
- It is deductive because **each work activity note in the WAAD** is treated as the **major premise** in a logical syllogism.
- The **second premise/ minor premise** is **everything you know about UX and interaction design**.
- The **conclusion** of this syllogism is a **statement of user needs and requirements you deduce from the work activity note**, something we capture in a “requirement statement.”

3. Preparation:

- Choose a requirements team leader and a recorder, a person experienced in writing requirements.
- You need a requirements “record” template in a word processing document, a spreadsheet, or a database schema to capture the requirement statements in a consistent and structured format in an interaction design requirements document.
- The requirements team will work in the room where the WAAD is posted on the wall.

4. Systematic Deduction of Needs:

- Identify needs from WAAD notes and use them as a basis for requirements.
- Example: A need for faster communication might lead to a design requirement for streamlined messaging tools.

5. Terminology Consistency:

- Standardize terms to avoid confusion and ensure clarity.
- Example: Use "notification" consistently instead of switching between similar terms like "alert" or "reminder."

6. Requirement Statements:

- Write requirements in a structured way with clear descriptions, rationale, and links to data sources.
- Example: "Ticket buyers shall be able to see a real-time preview of available seating for a venue."

7. Requirement Statement Structure:

- Use a consistent structure with major categories, rationale, and traceability tags (e.g., WAAD node IDs).
- Use headings, subheadings, and IDs to keep requirements organized and traceable.
- Example: "Security: Shall protect ticket-buyer privacy [C19]"

- **Requirement Statement Structure:**

- A requirements document is essentially a set of requirement statements organized on headings at two or more levels.

Name of major feature or category

Name of second-level feature or category

Requirement statement [WAAD source node ID]

Rationale (if useful): Rationale statement

Note (optional): Commentary about this requirement

Generic structure of a requirement statement.

Security

Privacy of ticket-buyer transactions

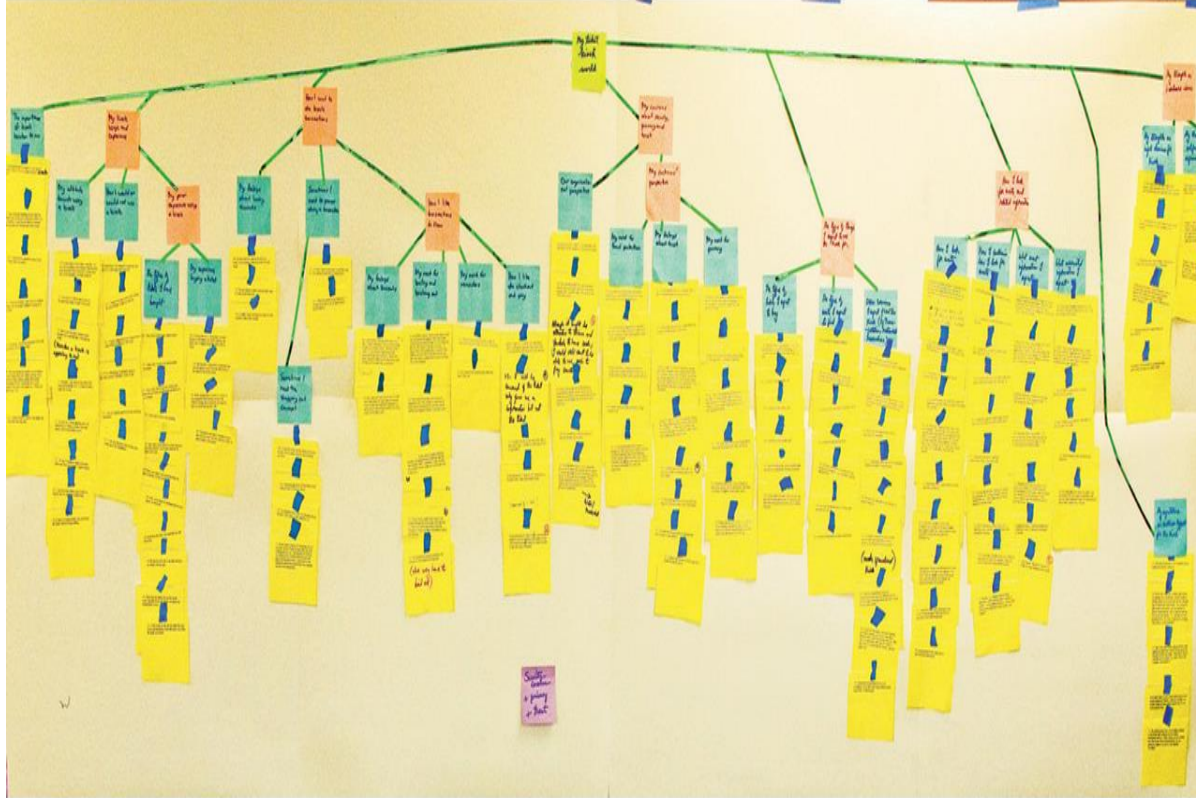
Shall protect security and privacy of ticket-buyer transactions [C19]

Note: In design, consider timeout feature to clear screen between customers.

Example requirement statement.

- We use A, B, C, ... for the highest-level nodes under the root node.
- Under node A, we use AA, AB, AC, ...
- For the work activity notes themselves we use the group ID plus a number, such as AB1 and AB2; this is the identifier that goes in the “WAAD source node ID” part of a requirement.

Example: Extracting a Requirement Statement for the Ticket Kiosk System



Sample WAAD for Ticket Kiosk System

Sample requirement statement for the Ticket Kiosk System.

Transaction flow

..... *Recommendations for buying*

Ticket-buyer purchases shall be supported by recommendations for the purchase of related items. [DE2].

Implied system requirement: During a transaction session the Ticket Kiosk System shall keep track of the kinds of choices made by the ticket buyer along with the choices of other ticket buyers who bought this item. [DE2].

Note: Amazon.com is a model for this feature.

8. Continue the Process for the Whole WAAD:

- Extract requirements from every relevant note in the WAAD.
- Review requirements as a team to ensure alignment and completeness.

9. Emotional Impact and User Experience Goals:

- Look beyond functional needs to address emotional and experiential aspects, like making the system enjoyable or visually appealing.
- Example: Address user frustrations noted in WAAD by designing a more intuitive interface.

10. Extrapolation Requirements:

- Broaden specific user needs into general requirements.
- Example: A user need for searching events may lead to a requirement for browsing and filtering options.

11. Other Outputs from Requirements Extraction:

- Identify missing data and gaps for further investigation.
- Highlight system needs like security, scalability, or reliability.
- Share useful user feedback with marketing teams.

12. Prioritizing Requirements:

- Categorize requirements based on importance with stakeholder input.
- Focus on high-priority needs first and consider lower-priority ones later.

13. Validating Requirements with Customers and Users:

- Review requirements with users and stakeholders to ensure accuracy and alignment.
- Example: Discuss with ticket buyers whether the timeout mechanism addresses their privacy concerns.

14. Addressing Organizational and Sociological Issues:

- Be aware of workplace dynamics and resistance to changes.
- Example: Introducing kiosks in a ticketing system may shift roles, like eliminating the ticket-seller role, requiring careful management of stakeholder expectations.

Design informing model extraction-second span of the bridge

- The **second span of the bridge** in the transition from contextual analysis to design focuses on constructing design-informing models.
- This stage involves **transforming the insights derived from contextual inquiry and requirements extraction into actionable artifacts** that guide and inspire the design process.
- Design-informing models are constructs (e.g., personas, task descriptions) that transform raw data into elements that inspire and guide design decisions.
- Design informing models **serve as a bridge between user research and the actual design phase, ensuring that the final product effectively meets the needs and expectations of its users.**

- Design informing models helps to:

- **Integrate and summarize the contextual data** : bring together large amount of data and complex data is simplified and organized so the designers can easily understand it.
- **Point back to the data**, to maintain the “chain of custody” to ensure that the design is based on real contextual data.
- **Provide a shared focus** for analysis now and, later, design.
- **Provide intermediate deliverables**, which can be important to your working relationship with the customer.
 - Outputs like personas, journey maps, task models acts as working documents.
 - Useful when communicating ideas and requirements to customers.

General "How To" Suggestions for Constructing Design-Informing Models

1. **Maintain Connections to Your Data** : This is to ensure that the design decisions should not be based on assumptions.
 - **Why:** Ensuring traceability between design decisions and raw contextual data enhances credibility and user-centeredness.
 - **How:**
 - Label all elements in the models with unique identifiers linked back to their original data sources.
 - Use line numbers, time codes, or WAAD (Work Activity Affinity Diagram) node IDs for reference.
 - Treat any item without a data reference as an unsupported assumption needing further scrutiny.

2. Extract Inputs Simultaneously with Requirements : identify design input and requirement at the same time during analysis,

- **Why:** This reduces redundancy and ensures consistency between requirements and models.
- **How:**
 - While walking through the WAAD or other contextual data to extract requirements, simultaneously identify inputs for design-informing models.
 - Look for references to user types, tasks, social concerns, and other design-relevant elements in the data.
 - Take notes on potential model components as they emerge naturally.

3. Use Sorted Bins of Work Activity Notes : organizing observations into categories by creating models

- **Why:**
 - Pre-sorted data simplifies the synthesis of models and aligns with anticipated modeling needs.
- **How:**
 - Organize work activity notes into categories or "bins" during contextual analysis (e.g., user-related notes, task-related notes).
 - Use these bins as inputs for creating corresponding models (e.g., personas, flow models, task descriptions).

4. Represent Barriers to Work Practice : shows problems and difficulties faced by user in their work processes.

- **Why:**

- Identifying barriers highlights opportunities for design improvements.

- **How:**

- Include problems and interruptions observed in workflows (e.g., communication breakdowns, coordination issues).
- Use symbols like red lightning bolts(⚡) to visually mark barriers in models.
- Address these barriers in the envisioned versions of the models.

- **Three Design-informing Models are:**

- User Model
- Usage Model
- Work Environment Model

USER MODELS

- User models are a set of models that define who the users are, including everything about work roles, sub-roles, user class definitions, and personas.
- It includes:
 - Work Roles
 - User classes
 - User personas

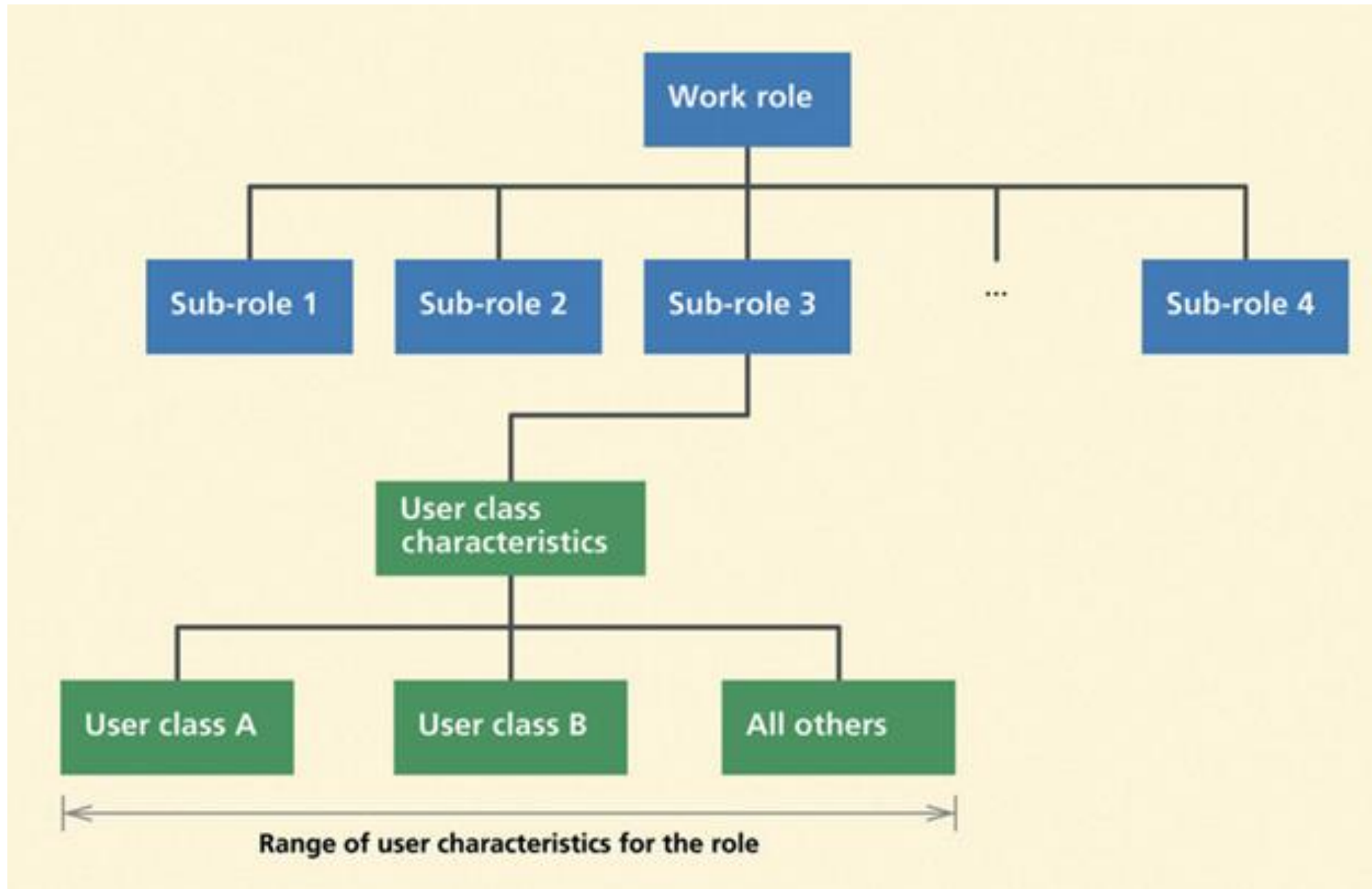
Work Roles

- A work role represent responsibilities and tasks tied to specific job titles or functions.
 - Example: A "ticket buyer" in a kiosk system who interacts directly with the machine to purchase tickets.
-
- **Sub-Roles** exist within broader roles, distinguishing specific tasks or responsibilities (e.g., student versus alumni ticket buyers in a ticketing system).
 - **Mediated Work Roles** involve users who interact with the system through intermediaries, like customers using a sales clerk.
 - **Envisioned Work Roles** are descriptions of how work roles might change with the introduction of a new system.

User Classes

- A user class is defined by a description of the relevant characteristics of people who might take on a particular work role.
- Every work role will have at least one user class description.
- Sometimes a work role can have such a broad user population that it requires more than one user class definition to describe all the different kinds of people who can assume that role.
- User class definitions document the general characteristics of these groups of people such as skills, knowledge, experience, training, education and special needs.
- For example, a medical doctor might be an expert in domain knowledge for an MRI system, but may have novice-to-intermittent knowledge in the area of related computer applications. In contrast, a secretary in the hospital may be a novice in the domain of MRI but may have more complete knowledge regarding the use of related computer applications.

Relationships among work roles, sub-roles, and user class characteristics.

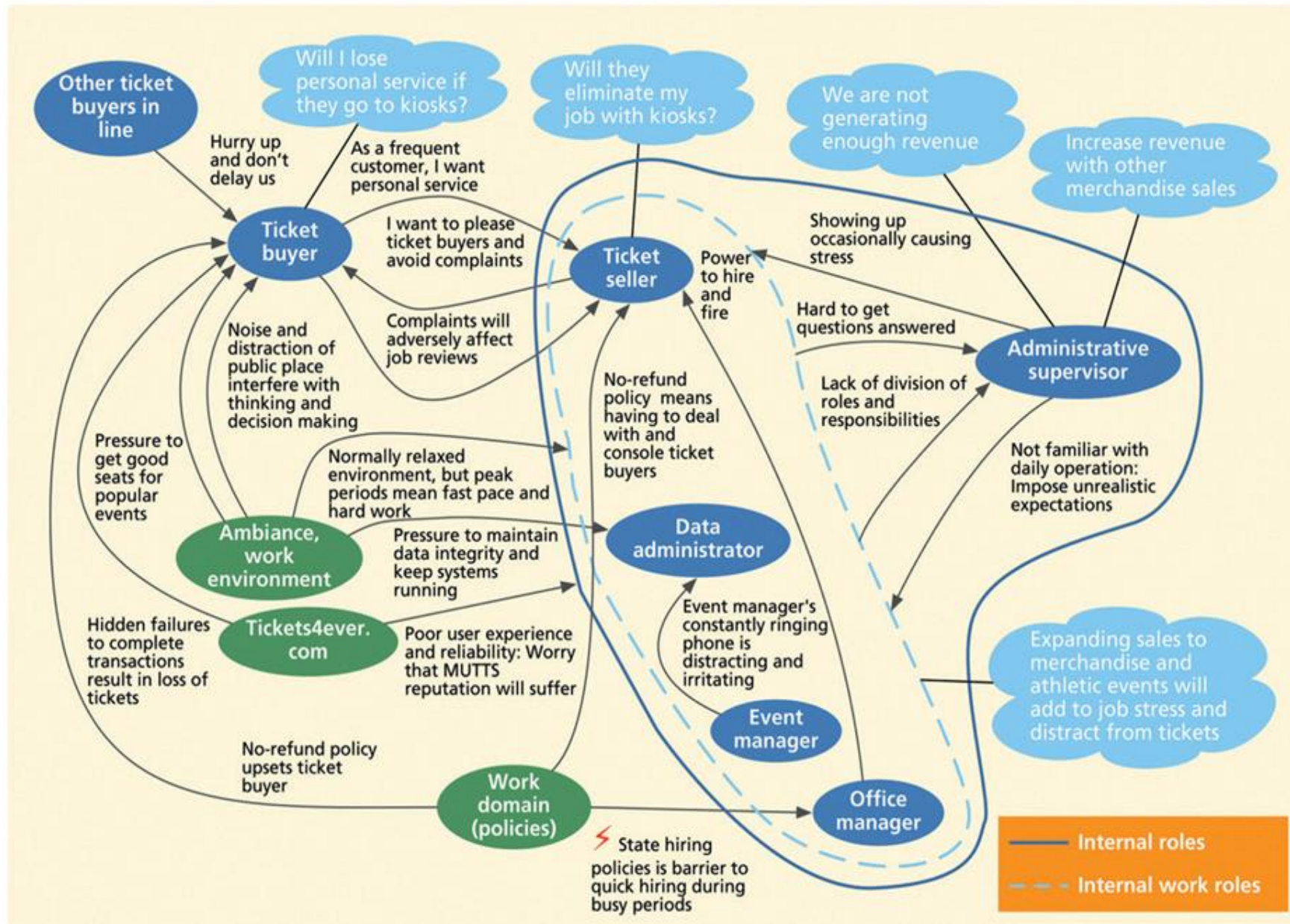


User Personas

- Personas are created based on data from user research and are typically detailed with attributes like age, job role, goals, frustrations, and behavioral patterns.
- Helps design teams visualize and empathize with users, ensuring that design decisions are centered around real user needs.
- **Example:** "John, a 40-year-old tech-savvy entrepreneur who values efficiency in mobile apps but gets frustrated by slow load times."

Social Models

- Representation of social dynamics and relationships between users within a system.
- They capture elements such as teamwork, communication patterns, and collaboration.
- The social model contains nodes connected with arcs. Nodes represent active roles and arcs represent social relationships, such as influence by role on another.



Ticket Seller:

- Directly deals with ticket buyers and ensures smooth ticket transactions.
- Manages complaints and issues related to the no-refund policy.
- Works under pressure during peak periods while maintaining customer satisfaction.

Event Manager:

- Oversees event logistics and ticket sales.
- Responds to operational challenges, often involving the ticket seller.
- Is responsible for communication with ticket sellers during high-stress scenarios.

USAGE MODELS

- Usage models are a set of models that define how work gets done in a system, focusing on the interactions between users, tasks, and workflows.
- Guide the design process by capturing the flow of activities and task relationships.
- Usage models are:
 - Flow models,
 - Task models,
 - Task structure model
 - Task interaction models
 - Information object model

Flow Model

- The flow model provides a high-level view of the work processes within a domain.
- It visually represents the workflow, including interactions between various work roles and system components.
- It helps us understand how information or action flow through an organization or process.
- Key elements include:
 - **Nodes**: Represent entities involved in the workflow, such as people, devices, and systems.
 - **Arcs (Arrows)**: Indicate the flow of information or actions between entities.
- The flow model is crucial because it shows how information moves across an organization and reveals communication patterns.
- It is often sketched early during contextual inquiry and refined throughout the design process.

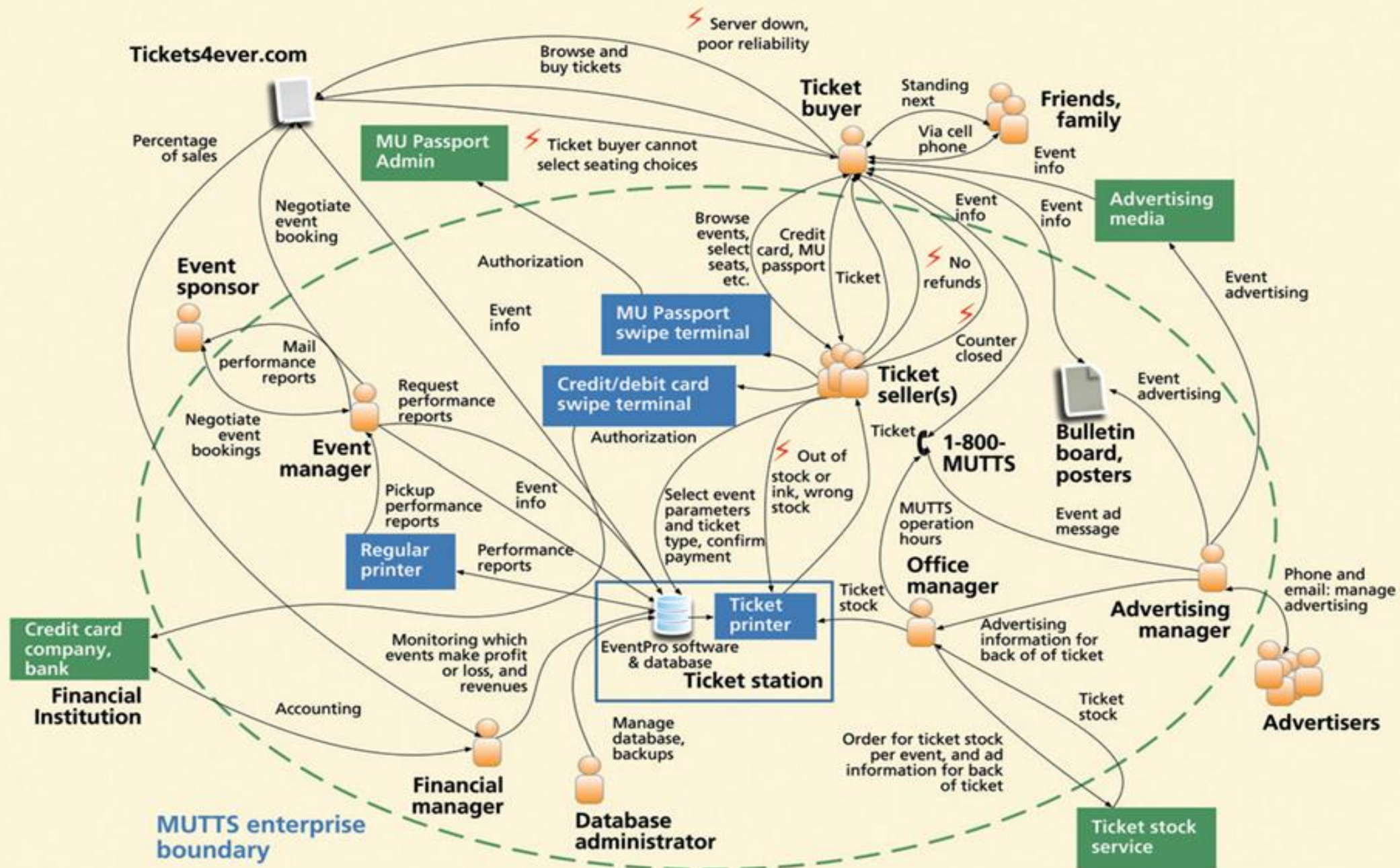
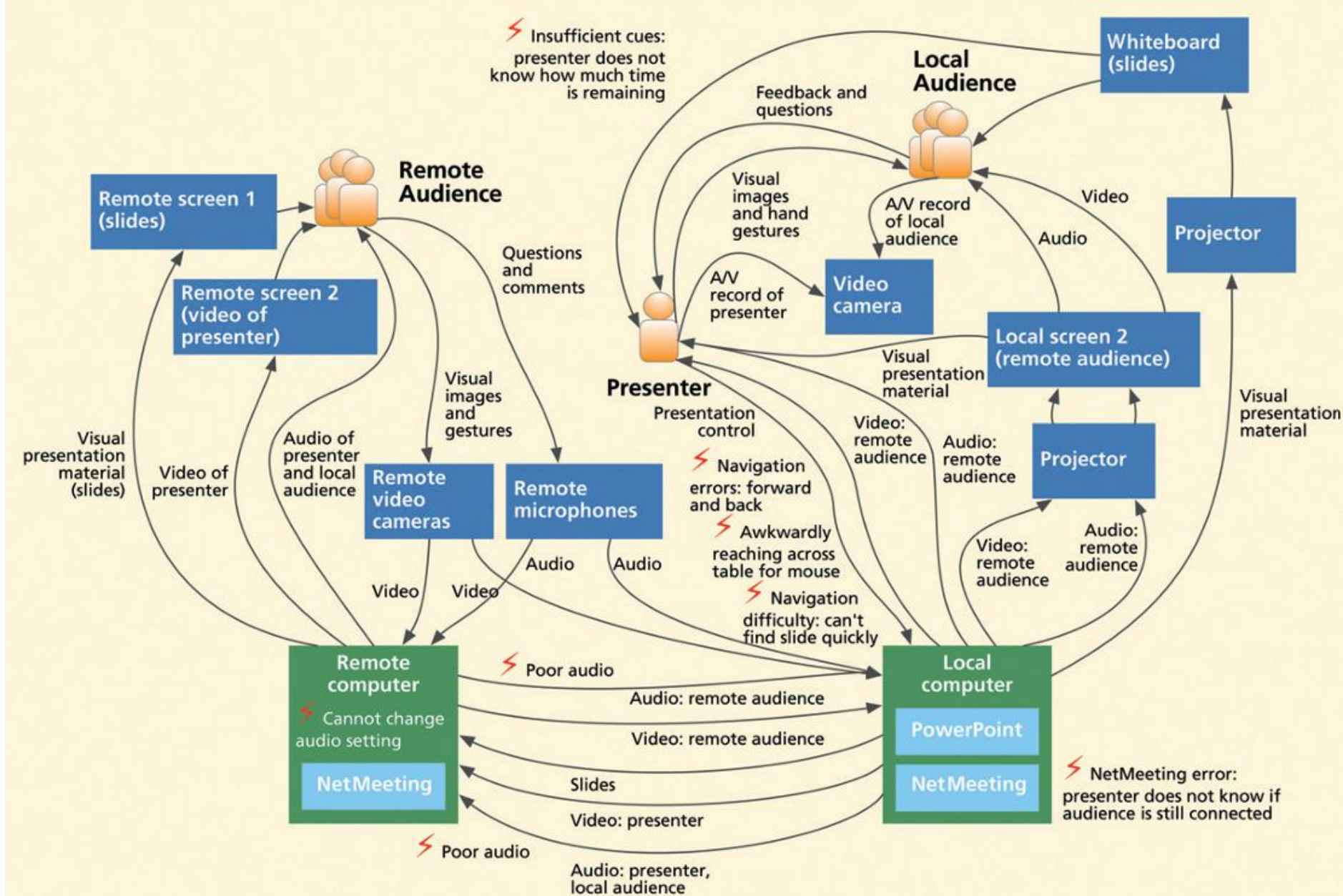


Fig: Flow model for MUTTS

Example flow model from the slideshow presentation contextual inquiry.

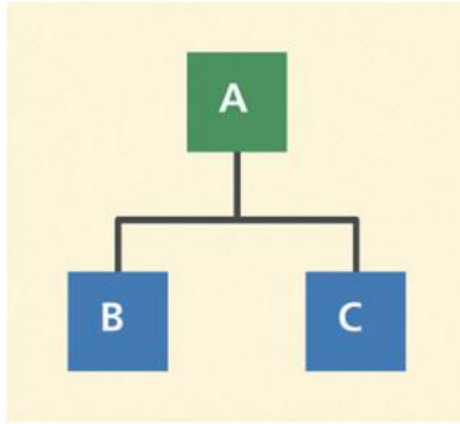


Task Models

- A Task Model describes how users perform tasks when interacting with a system.
- It breaks down tasks into smaller steps, showing the sequence of actions needed to achieve a goal.
- Key Components:
 - **Tasks**: The main actions a user performs (e.g., "Buy a ticket").
 - **Subtasks**: Smaller steps within a task (e.g., "Select event," "Choose seats," "Confirm payment").
 - **Order and Flow**: The sequence in which tasks and subtasks occur.
 - **User Goals**: What the user aims to accomplish (e.g., successfully purchasing a ticket).

❖ Task Structure Model

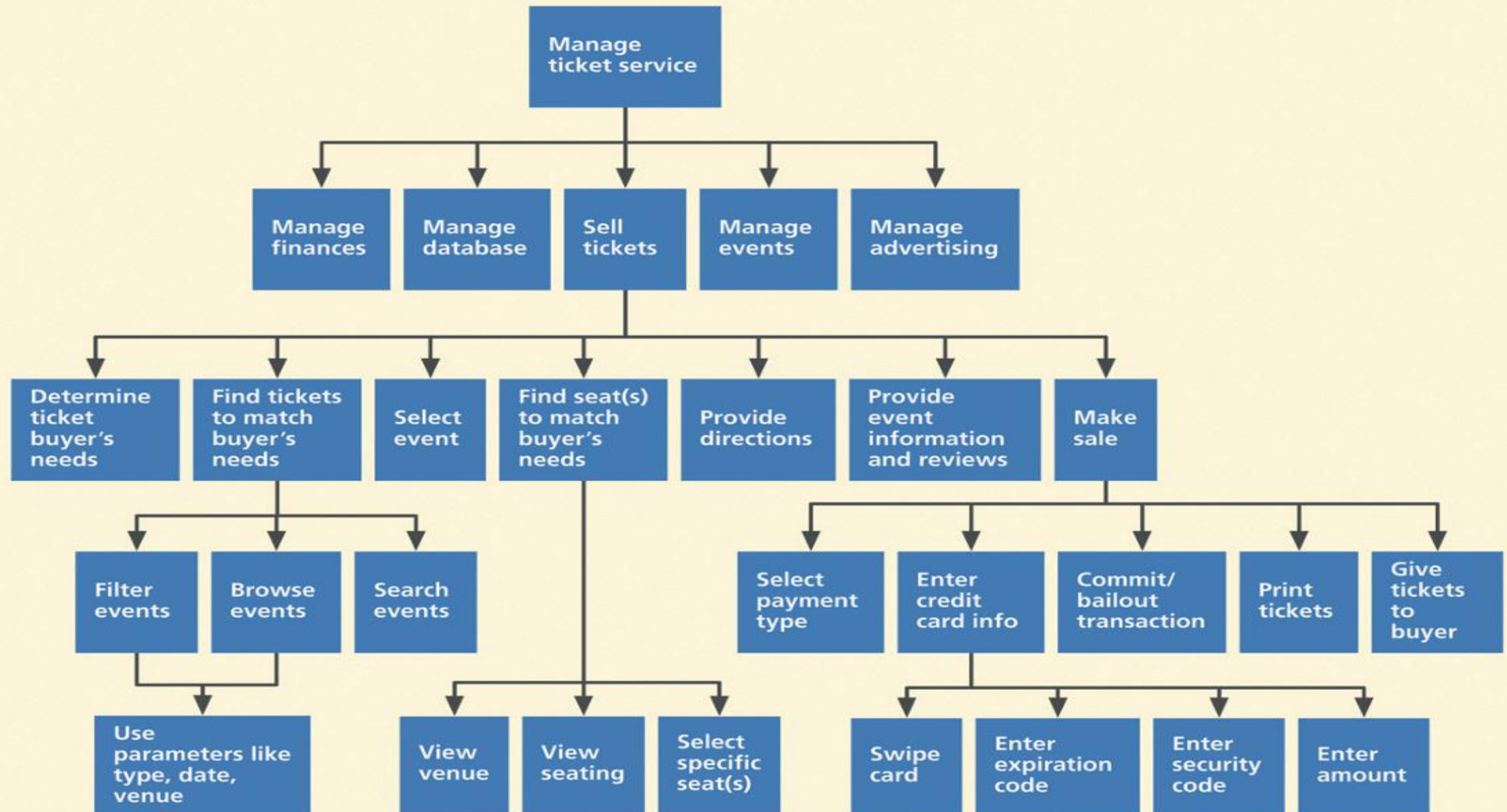
- This model breaks down a user's goal into smaller tasks and subtasks, showing the logical flow and hierarchy of actions needed to complete that goal.
- It visually organizes tasks in a structured way to help designers understand how users interact with a system.
- A hierarchical task inventory (HTI), in which tasks are broken down into a series of subtasks and steps, is used.
- **Key Components:**
 - **Goal:** The main objective the user wants to achieve (e.g., "Buy a Ticket").
 - **Tasks:** The major steps required to achieve the goal (e.g., "Search for an Event").
 - **Subtasks:** Smaller steps within each task (e.g., "Enter event name," "Filter by date").
- **Order/Sequence:** The logical flow of tasks and subtasks, often shown as a hierarchy.



If task A is drawn above task B, it means A is a super-task of B, or B is a subtask of A. Exactly the same relationship exists between A and C. B and C are “sibling” tasks.

Hierarchical relationship of task A, the super-task, and tasks B and C, subtasks.

Partial HTI for MUTTS
“sell tickets” task.



- ❖ **Task Interaction Model**

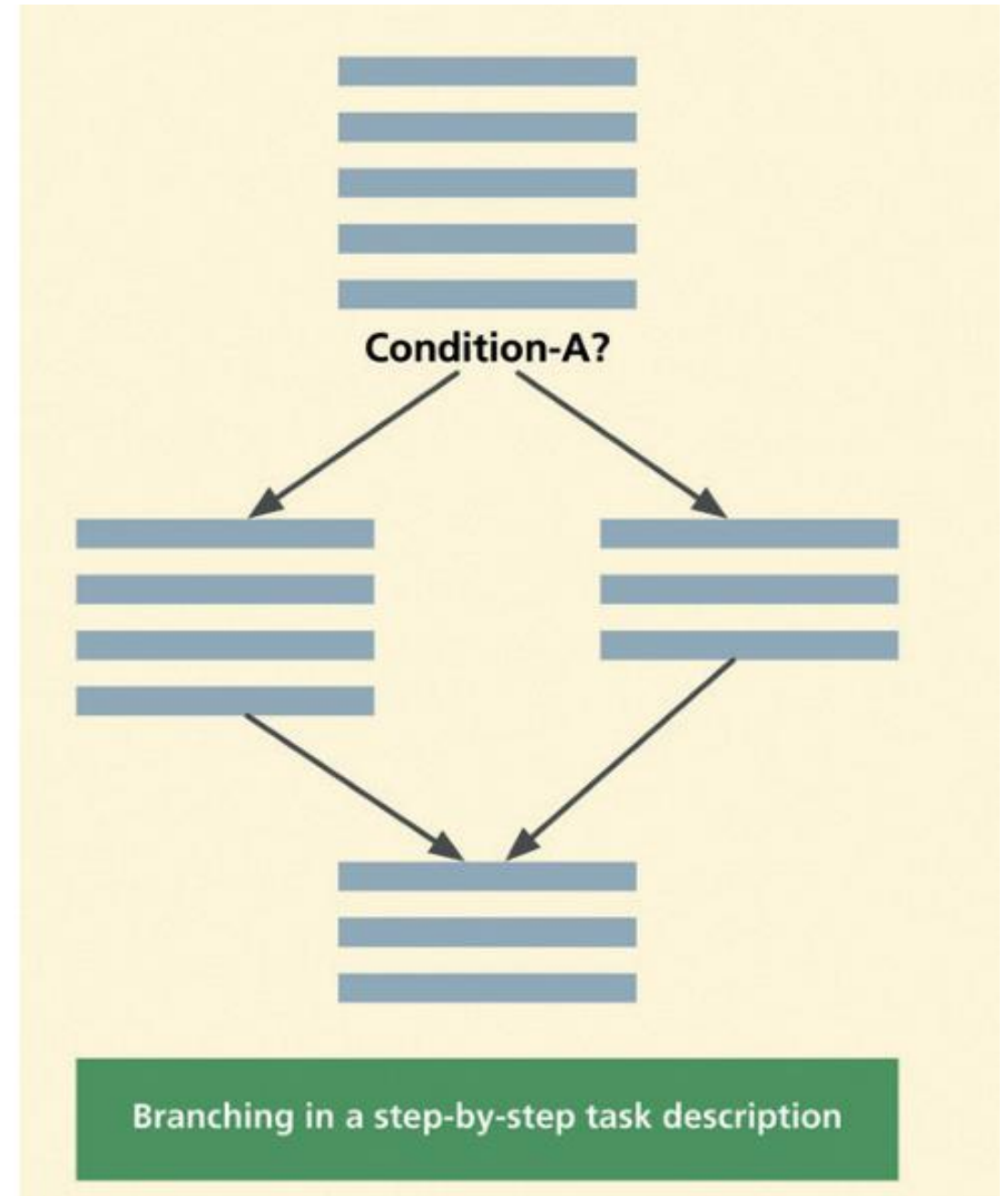
- This model describes how users interact with a system step by step while performing a task.
- It focuses on the dynamic relationship between the user actions and system responses, showing the flow of interaction during task completion.
- **Key Components:**
 - **User Actions:** Steps the user takes (e.g., clicking a button, entering data).
 - **System Responses:** How the system reacts to user actions (e.g., displaying a confirmation message).
 - **Sequence:** The order in which actions and responses occur.
 - **Feedback:** System messages that inform the user of progress or errors.

- **Step-by-step task interaction models:**

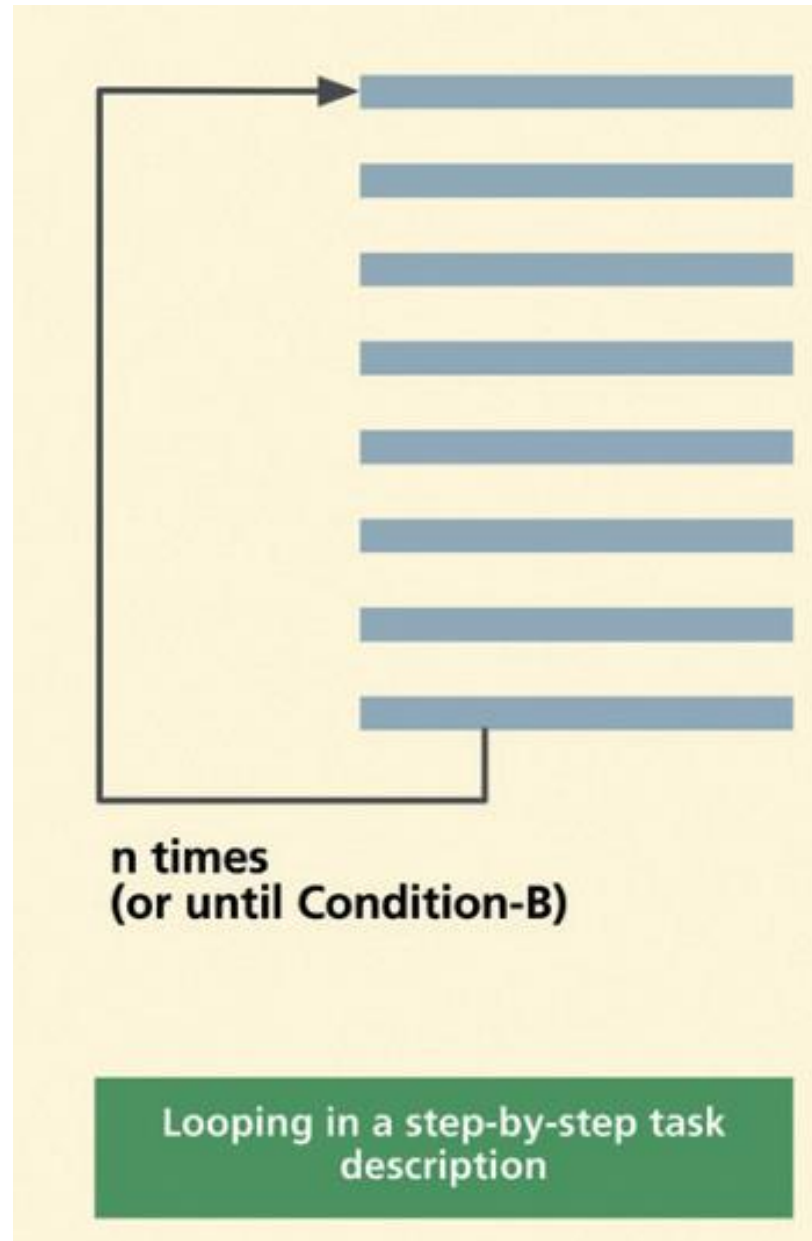
- A step-by-step task interaction model contains a **detailed description of task performance observed in users or as told by users.**
- At the beginning, individual task interaction models will be mostly linear paths without branching. Later you can add the most important branching or looping.
- **Task and step goals:** A task or step goal is the purpose, reason, or rationale for doing the task or taking the step.
- **Task triggers:** A task trigger is an event or activation condition that leads that user to initiate a given task.
- **Information and other needs in tasks:** One of the most important components of a task description is the identification of user information and other needs at any step.
- **Barriers within task interaction models:** difficulties encountered that effects task performance, including things that slow the user down and make a task more difficult than necessary.
- **Creating a step-by-step task interaction model:** Step-by-step task interaction models are mostly textual. Write the initial task interaction model as a linear task thread, as a model of one instance of how a task happened with a user, not a general model of how all users perform the task. Sequential steps can be written as an ordered list without the need for flowchart-style arrows to show the flow. Linear lines of text are less cluttered and easier to read.

- **Branching and looping:**

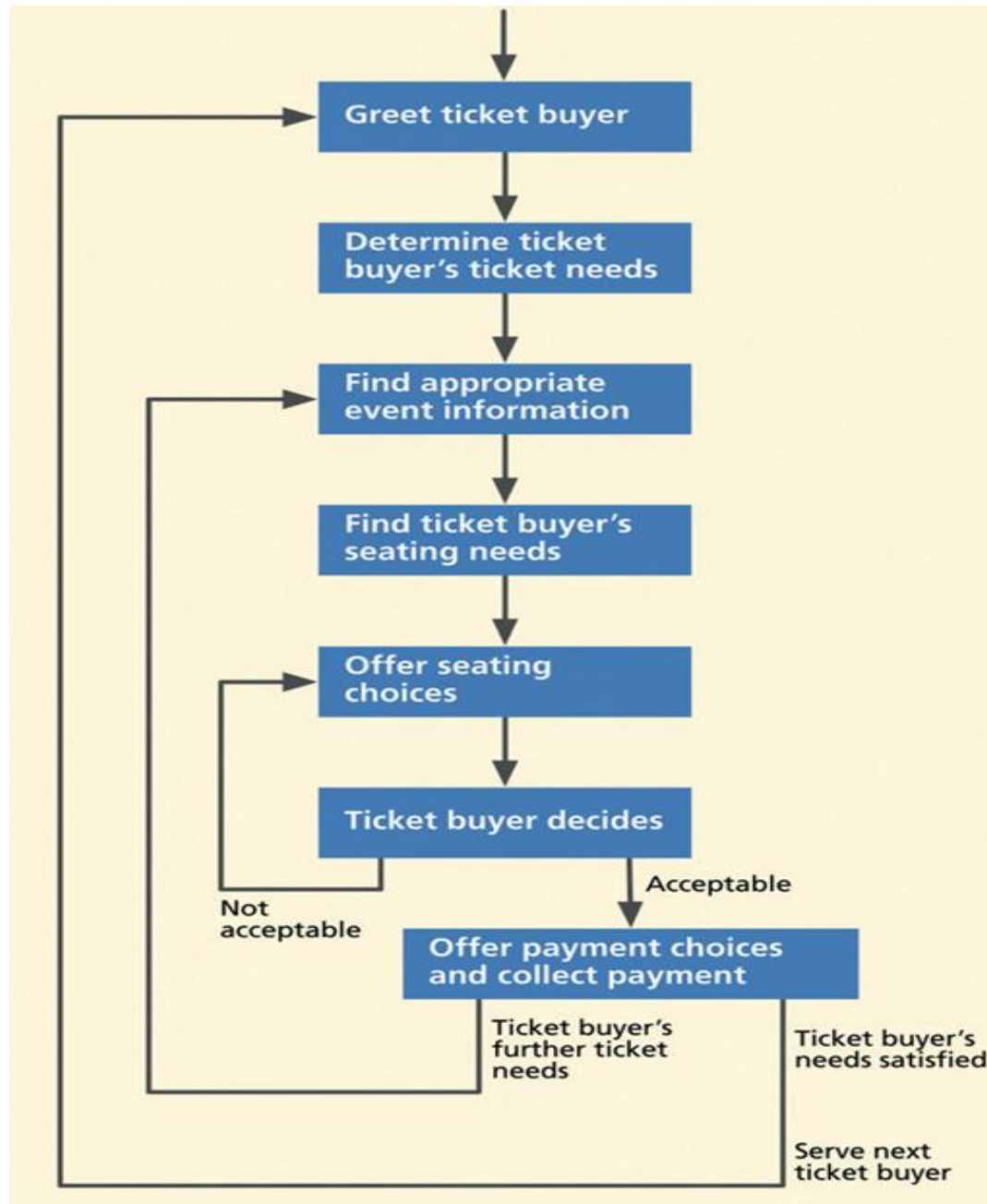
- Although step-by-step task interaction models are primarily for capturing linear sequences of representative task steps, sometimes you encounter a point in the work practice where there is a choice.
- You observe some doing A and other users B.
- You can generalize the task sequence representation by showing this choice in both observed paths using branching.



- Similarly, if you observe iteration of a set of tasks or task steps, you can represent that as



Task interaction branching and looping for MUTTS.



Information object model

- This model describes the **key objects or data elements within a system that users interact with.**
- It focuses on **how these objects are structured, their attributes, and how they relate to each other and the tasks users perform.**
- Key Components:
 - **Information Objects:** Core items users interact with (e.g., a "Ticket" in a ticketing system).
 - **Attributes:** Properties or details about the object (e.g., a ticket's event name, date, price).
 - **Relationships:** How objects relate to each other (e.g., a "Ticket" is linked to an "Event").
 - **Actions on Objects:** What users can do with the objects (e.g., search, modify, delete).

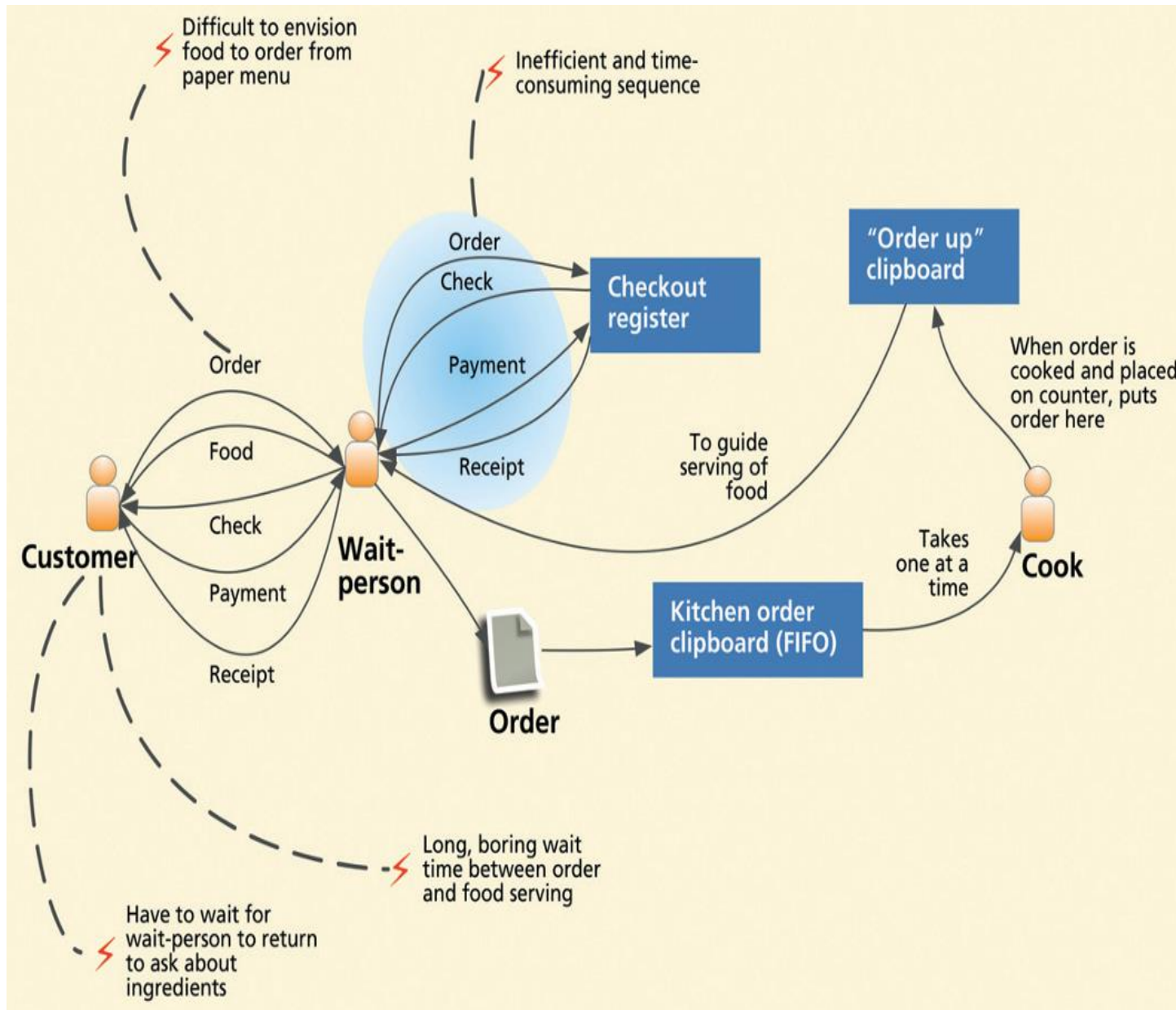
WORK ENVIRONMENT MODEL

- This model describes the **physical and contextual factors influencing how tasks are performed in a workspace.**
- It **focuses on the environment where work occurs**, capturing elements like physical constraints, tools, and workspace layout.
- Helps design systems that fit naturally into users real work space.
- It includes:
 - **Artifact Model**
 - **Physical model**

❖ Artifact Model

- This model represents the tangible elements—both physical and digital—that are used, shared, or modified during work activities.
- These artifacts help reveal work practices and how information is handled during task completion .
- **Key Components:**
 - **Artifacts:** Tangible objects such as forms, documents, tools, or digital files (e.g., a restaurant menu, an order form, or a digital spreadsheet).
 - **Attributes:** Properties of the artifact (e.g., fields in a form or labels on a chart).
 - **Usage Patterns:** How the artifact is used, shared, or modified in the workflow.

Eg: Part of a restaurant flow model with focus on work artifacts derived from the artifact model.



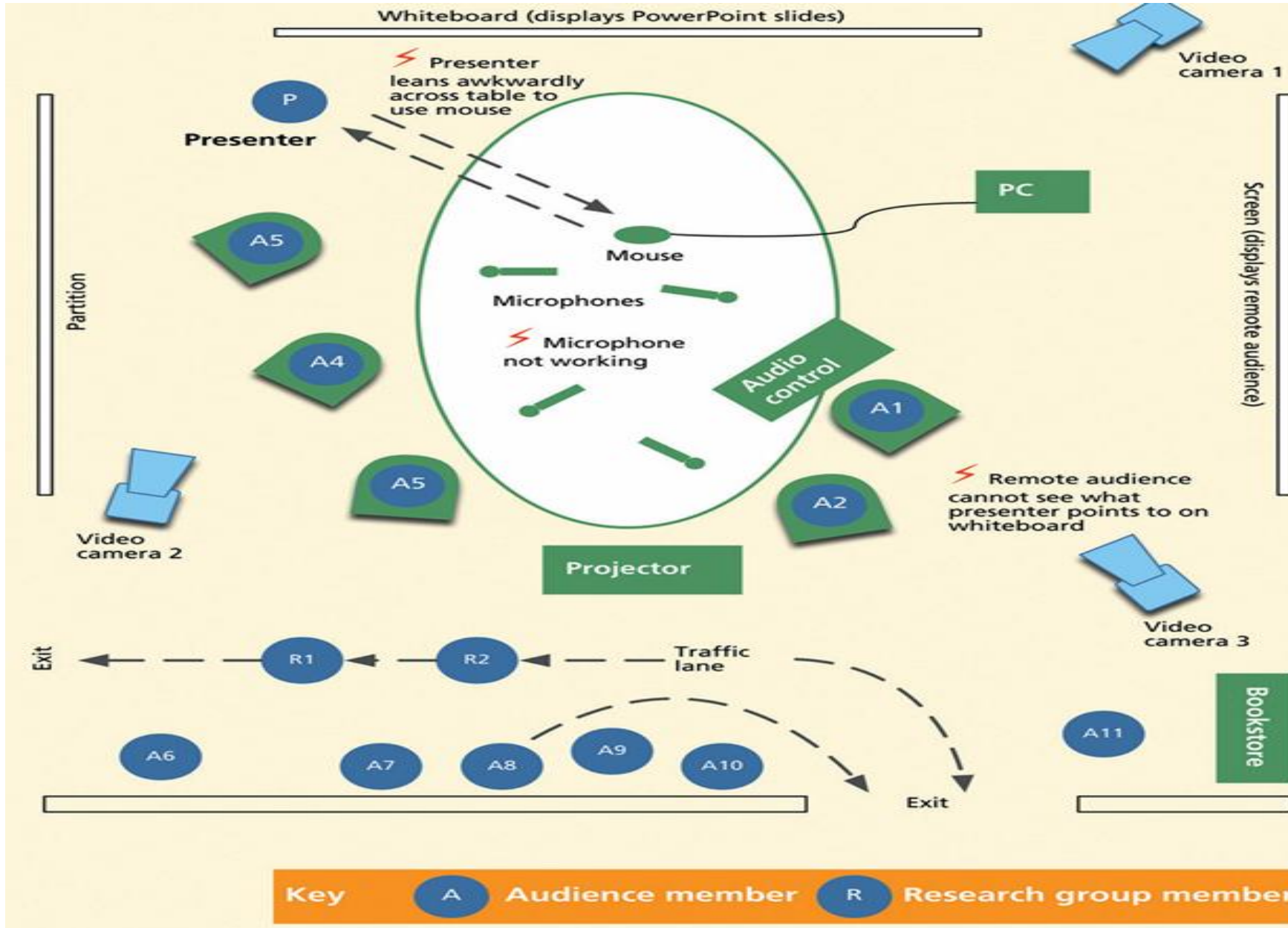
Constructing the artifact model

- In contextual inquiry you will have **collected the artifacts**.
- The team should make **"artifact posters."**
- **Attach samples of each artifact to a separate flip chart page.**
- **Add annotation to your "exhibits"** to provide more information about your observations of them in the work practice.
- **Add explanations of how each artifact is used** in the work practice and workflow.
- **Annotate artifacts with stick-on notes** associating them with tasks, user goals, and task barriers.
- The process can generate additional user work activity notes from what is learned about the artifacts and how they are used.

❖ Physical Model

- This model describes the physical layout and environment where tasks are performed.
- It shows how people, objects, and equipment are arranged within a workspace and how their physical positioning affects task efficiency and collaboration .
- **Key Components:**
 - **Physical Space:** The layout of rooms, desks, and physical structures.
 - **Work Equipment:** Devices like computers, printers, and telephones.
 - **Artifact Placement:** Locations of tools and documents related to tasks.
 - **Movement Paths:** How people move within the space, showing potential barriers or inefficiencies.

Eg: physical model for slideshow presentation



- This diagram represents the physical setup of a slideshow presentation, highlighting various elements such as the presenter's position, audience seating, research group members, equipment placement, and potential issues in the presentation environment.

BARRIER SUMMARIES

- Barrier summaries help **consolidate issues identified during contextual inquiry and analysis** that disrupt user work practices.
- These barriers can **interfere with task performance, delay actions, or create inefficiencies**, making it critical to address them for effective system design.
- **Example (Slideshow Presentation System):**
 - A study analyzing slideshow presentations identified common barriers such as:
 - **Awkward slide control placement** requiring presenters to walk back and forth between the audience and the control station.
 - **Audio issues** where remote participants couldn't hear the presenter clearly.
 - **Difficulties finding previous slides**, causing interruptions in task flow.

MODEL CONSOLIDATION

- Model consolidation involves **merging and unifying multiple models of the same type, often created by different sub teams during contextual analysis.**
- The **goal is to create a single, generalized model** by combining insights and eliminating redundant or conflicting details.
- **Key Steps in Model Consolidation:**
 1. **Identify Overlapping Elements:** Different sub teams may model the same work roles or tasks with slight variations. The first step is to identify these overlaps.
 2. **Merge and Unite Models:** Combine the similar parts while factoring out unnecessary differences to create a comprehensive representation.
 3. **Generalize Data:** Move from specific instances to a more abstract representation that captures common patterns across multiple data sources.

PROTECTING YOUR SOURCES

- One of the things to watch out for throughout the process, especially when dealing with design-informing models, is **confidentiality**.
- Protecting the identity of people or sources who provide the information helps build trust.
- If the sources are not protected, it could cause trouble or risk for them.
- When breakdown occurs, revealing sources can be dangerous because individuals or group might face blame or consequences.

ABRIDGED METHODS FOR DESIGN-INFORMING MODELS EXTRACTION

- Abridged methods are streamlined techniques for extracting design-informing models, focusing on efficiency while still capturing essential insights.
- These methods are particularly useful for experienced designers and projects with limited time or resources.

- Key Techniques:

1. Selective Modeling:

- Not all models need to be developed in full detail.
- Choose models based on the work domain and design goals.
- Example: If physical space is irrelevant, a physical model can be skipped.

2. Designer-Ability-Driven Modeling:

- The depth of modeling depends on the designer's experience and familiarity with the domain.
- Less experienced designers might need detailed models for clarity, while experienced ones can rely on mental abstractions.
- Risk: Simplifying too much can lead to missed details and usability issues.

3. Use a Hybrid of WAAD and Relevant Models

- Combine Work Activity Affinity Diagram (WAAD) with elements of other models on a single workspace.
- This allows for visual representation of work patterns, so the designers can understand easily.
- It reduce the need for many separate formal models, saving time and effort.

4. Create Design-Informing Models on the Fly during Interviews

- Create models during user interviews or contextual inquiries.
- Capture models as rough sketches while observing and speaking with users.
- This helps designers understand work processes accurately.
- Example: Sketching a flow model while observing how a ticket seller handles transactions.

ROOTS OF ESSENTIAL USE CASES IN SOFTWARE USE CASES

- Essential use cases originate from traditional software use cases but focus more on capturing task goals and user intentions rather than system functionality.
- They were introduced as a response to the limitations of standard software use cases, especially when addressing interaction design requirements.
- Traditional software use cases emphasize system behavior, describing actors, system responses, and event sequences, but often neglect user experience considerations .
- Essential use cases shift focus from how a system works to what a user wants to achieve, making them ideal for user-centered design.